



BooN: Boolean Network Analysis Software

Franck Delaplace^{1,*}

¹IBISC, Paris-Saclay University, Evry, France

*Corresponding author. franck.delaplace@univ-evry.fr

FOR PUBLISHER ONLY Received on Date Month Year; revised on Date Month Year; accepted on Date Month Year

Abstract

In this paper, we present an open source software, called **BooN**, which has been developed for the purpose of Boolean network analysis in Python. The release includes two main packages: one comprising a set of methods applied to the Boolean network object and the other for the graphical interface. The software encompasses all the requisite features for Boolean network analysis, including graphical network design, interaction graph drawing, model of dynamics, symbolic computation of stable states, and controllability analysis.

Availability and Implementation

BooN is freely available on [GitHub](https://github.com/Franck-Delaplace/BooN) at <https://github.com/Franck-Delaplace/BooN>.

Key words: Boolean Network, Controllability, Software

1. Introduction

Boolean Network Analysis applied to biological systems studies was pioneered by Stuart Kaufman(Kauffman, 1969; Glass and Kauffman, 1973) and René Thomas(Thomas, 1973) in 70s. The objective was to develop a formal modeling framework for studying the dynamics of genetic networks, considering the available data for their analysis.

Currently, this modeling framework is regarded as the gold standard for studying biological systems, as evidenced by the substantial number of publications on biological network and system modeling (Figure 1). Notably, these publications have significantly surpassed other modeling approaches in terms of prevalence.

One of the most significant challenges for this modeling framework concerns the development of software platforms facilitating their use by a large number of users with a diverse range of skill sets. Such platforms should offer a comprehensive range of functionality and a user-friendly graphical interface.

Several software solutions provide a computational framework for Boolean network analysis. These are available in a variety of programming languages and offer a range of functions for performing this analysis.

GINSIM (Naldi et al., 2009) provides an interactive design of the Boolean network, including annotations using a GUI. It offers a step-wise simulation and full dynamics for any mode, as well as a computation of stable states based on symbolic analysis and a full description of attractors from a brute force algorithm limiting its application to small network. It has been developed in JAVA.

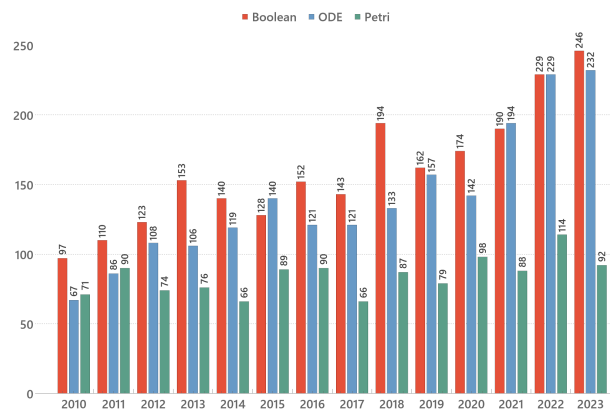


Fig. 1: Number of publications per modeling framework (Boolean network, ODE, Petri Net) on PUBMED.

BoolNet(Müssel et al., 2010) is a R-package also devoted to Boolean network analysis. It tackles with the synchronous and asynchronous analysis and also probabilistic Boolean network.

It includes functions for various simulations, attractor search, and robustness analysis but does not contain symbolic algorithm for stable states computation. The specification of a Boolean network is achieved from formula description. BoolNet is reduced to a

package and does not contain a graphical interface for facilitating the network design.

Similarly PyBoolnet(Klärner et al., 2017) is a Python package devoted to Boolean network analysis including some graph facilities.

The Python package AON (Beneš et al., 2022) is another tool devoted to the analysis of asynchronous dynamics. It allows the examination of partially specified Boolean networks with uncertain update functions. The package is based on the binary decision diagram, which enhances the attractor detection algorithms.

BoolSi(Oles and Kukushkin, 2020) is an open-source cross-platform command line tool restricted to simulations of synchronous Boolean networks. It supports parallel processing using MPI. BoolSi is used to model the behavior of complex dynamic networks and allows for identification and statistical analysis of network attractors.

This article introduces **BooN** a new Python-based platform that provides a comprehensive set of functionalities for Boolean network analysis, including the controllability analysis, and a user-friendly graphical interface for Boolean network manipulation.

Section 2 recalls the main features of a Boolean network analysis including equilibria computation and controllability analysis. Section 3 details the different functionalities of **BooN**. Section 4 presents a benchmark analysis of the primary methods of **BooN**, with the objective of quantifying the time required for the application of each method to random Boolean networks. In conclusion (Section 5) we compare **BooN** to software related to Boolean network analysis.

2. Boolean Network

2.1. Basic definitions

A *Boolean network* is a discrete dynamical system operating on Boolean variables X . It is defined as a system of Boolean equations of the form: $x_i = f_i(x_1, \dots, x_n), 1 \leq i \leq n$ where each f_i is a logical propositional formula. We present in Figure 2 the dynamics of the following Boolean network:

$$F = \begin{cases} x_1 = (x_1 \vee x_2) \\ x_2 = (\neg x_3 \wedge x_1) \\ x_3 = (x_2 \wedge x_1) \end{cases} \quad (1)$$

A Boolean *state* of s is an interpretation of the variables of X in Boolean *i.e.*, $s : X \rightarrow \mathbb{B}, \mathbb{B} = \{0, 1\}$, and $S_X = (X \rightarrow \mathbb{B})$ is the set of all states.

The analysis of a Boolean network is primarily focused on the dynamics to investigate its behavior. In contrast to simulations conducted within other modeling frameworks, such as ODEs, the Boolean dynamics provide the full description of all potential trajectories, thereby ensuring a comprehensive and thorough investigation of all possible dynamical scenarios.

Formally, a *model of dynamics* is defined by a labeled transition system where for each transition the states of agents are updated with regard to an updating policy defined by a *mode*. Usually, two modes are used: the *asynchronous* mode where one variable exactly is updated per transition and the *synchronous* mode where all the variables are updated per transition. As example, the *asynchronous* mode leads to the following labeled transition system $\langle \rightarrow, X, \mathbb{B}^n \rangle$ where the updated agent $\rightarrow \subseteq S_X \times X \times S_X$ labels the transition relation, $\xrightarrow{x_i}$ such that:

$$s_1 \xrightarrow{x_i} s_2 \stackrel{\text{def}}{=}$$

$$s_1 \neq s_2 \wedge s_2(x_i) = f_i(s_1) \wedge \forall x_j \in X \setminus \{x_i\} : s_2(x_j) = s_1(x_j).$$

A key objective of the analysis is to assess the equilibrium of the modeled network to gain insight into its long-term behavior. Basically, a state s is an *equilibrium* for $\rightarrow$, if it can be infinitely reached once met, *i.e.*, $\forall s' \in S_X : s \rightarrow^* s' \implies s' \rightarrow^* s$. The aim is to identify and characterize the attractors, which are defined as sets of mutually reachable equilibria.

The attractors corresponding to a single state are called *stable states* and can be efficiently and quickly computed by symbolic method while the computation of the other attractors (cyclic) remains exponential in general, limiting their investigation to small networks.

Finally, an *interaction graph* $\langle X, \rightarrow \rangle$ portrays the causal interactions between variables of a Boolean network. An interaction $x_i \rightarrow x_j$ exists if and only if x_i occurs as literal in a minimal Disjunctive Normal Form (DNF) of f_j , *i.e.*, $x_i \rightarrow x_j \stackrel{\text{def}}{=} x_i \in V(\text{DNF}(f_j))$. The interactions are refined by a sign stipulating the nature of the interaction: positive (+) or negative (-). Figure 2 illustrates the asynchronous dynamics of Boolean network (1) leading to a cyclic attractor (red) and a stable state (yellow), and its signed interaction graph.

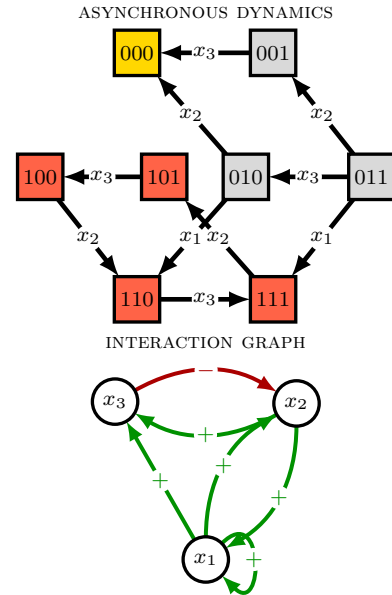


Fig. 2: Asynchronous dynamics and equilibria (in different colors for each attractor), interaction graph

2.2. Controllability

Recently, a significant body of research has been devoted to the study of controllability in Boolean networks. The aim of this research is to identify the necessary modifications to the equations governing the network to figure out the means of satisfying a query at a given equilibrium state. The theoretical framework, and the detail of the computational methods developed in **BooN** for the

controllability analysis can be found in (Biane and Delaplace, 2019) and (Pardo et al., 2021) for its extension to control sequence.

Adding control extends the Boolean network to *Boolean control network* (BCN) where the *control parameters* u_i are added to the equation for figuring the control action. The classical modifications of equations consist in freezing a variable by setting it to True/1 or False/0. Basically, adding control extends the formula to include the control parameters. A freeze occurs when the control parameters freeze ($u_i = 0$) otherwise it is inactive ($u_i = 1$). The formalization of the control extension can be achieved as follows for each equation:

$$\begin{aligned} x_i &= f_i(x_1, \dots, x_n) \wedge u_i^0 && \text{for freezing to 0} \\ x_i &= f_i(x_1, \dots, x_n) \vee \neg u_i^1 && \text{for freezing to 1.} \end{aligned}$$

Note that when $u_i^0 = 0$ then the equation becomes $x_i = 0$ and similarly when $u_i^1 = 0$, $x_i = 1$.

The Boolean network (1), subjected to control thus becomes:

$$\begin{cases} x_1 = (x_1 \vee x_2) \wedge u_1^0 \vee \neg u_1^1 \\ x_2 = (\neg x_3 \wedge x_1) \wedge u_2^0 \vee \neg u_2^1 \\ x_3 = (x_2 \wedge x_1) \wedge u_3^0 \vee \neg u_3^1 \end{cases} \quad (2)$$

The BCN has now 6 control parameters.

With this method, the control is applied on nodes because the variable is set to a Boolean value when the control freezes. The control can be also more selectively applied on links (Biane and Delaplace, 2019). As the Boolean network is modified when the control freezes, the dynamics is altered leading to different equilibria.

The resolution of control seeks to identify the active control required to fulfill a specified query. This query articulates a condition related to stable equilibria, which may take the following forms: *possibility*, signifying the existence of at least one stable state that satisfies the query, or *necessity*, mandating that all stable equilibria must conform to the query. The two forms of queries are included in **BooN**'s controllability analysis.

To illustrate this situation, if the aim is to reach $x_3 = 1$ at a stable state, the sole control that will achieve this outcome is to activate u_2^1 i.e., $u_2^1 = 0$ while $u_i^c = 1$ for the other configurations, the resulting BCN and its dynamics are shown in Figure 3. Now only one new stable state exists (111).

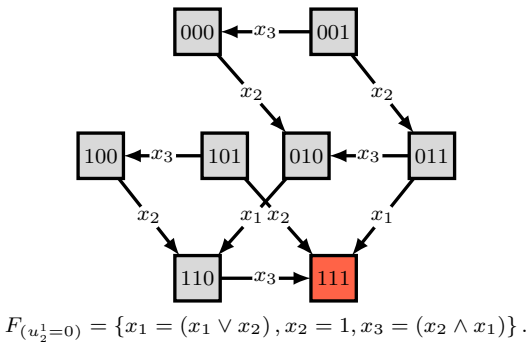


Fig. 3: Controlled network of F by assigning $u_2^1 = 0$.

The primary challenge is to accurately determine the control parameters' freeze actions to regulate the dynamics to achieve the

desired equilibrium. As previously referenced, a variety of methods have been developed to address this issue.

Controllability analysis is a valuable investigative tool in drug target discovery and disease etiology. Typically, a biomarker profile identifies a specific state, either disease or healthy. From control analysis, we can determine which variables should be turned ON or OFF, that is frozen, in order to achieve the desired binary biomarker profile at stable state. This allows us to properly identify the actions that should be applied to genes, either over-expression or inhibition. In the distribution (see [GitHub](#) site), we have fully described an example by a Jupyter program illustrating the use of controllability analysis for the identification of the causes of diseases and the therapeutic target discovery applied to breast cancer. This analysis is also described in the README for its interactive application.

3. BooN

BooN facilitates the analysis of the Boolean network including the network design, the equilibrium computation and the controllability analysis.

BooN has also been added in Colomoto (Naldi et al., 2018) repository collecting software related to Biological network analysis. The distribution includes the sources, complete documentation of the different methods applied to **BooN** object, tutorials and various examples including a real application of the controllability analysis on Breast cancer leading to drug target prediction. **BooN** is developed in Python and composed of 3 packages:

- **boon** is the core library providing a comprehensive set of methods for manipulating the **BooN** object.
- **logic** is a sub-library of **boon** containing the basic functions related to propositional logic, notably: basic functions, formula pretty printing, and prime implicants computation.
- **boonify** is the user-friendly graphical interface (GUI) interfacing the **boon** function based on PyQT5.

BooN has an internal file format for the storage of Boolean networks. However, for interoperability with other applications, Boolean networks in BoolNet, SBML, or Python-based format can also be imported.

Figure 4 illustrates the full range of analysis features accessible via different windows using the GUI for Boolean network (1).

Concerning the GUI, the design of the Boolean network is completely achieved from the drawing of the interaction graph (Figure 4). From the interaction graph drawing a DNF formula is generated. Therefore there is no conceptual limitation on the formulas since all formulas can be converted to DNF. The user draws the positive or the negative interactions and specifies in which module the variable belongs. A module characterizes a set of cooperative variables. It is identified by an integer id and translated into an AND-clause in the DNF formula. Also note that the formula can be modified directly using the Formula View.

Moreover, the different analysis (Dynamics w.r.t. a mode, Controllability, symbolic Stable state computation) are accessible via a menu and buttons located in the both sides of the main window where the interaction graph is drawn. Comprehensive online help is also available.

More technically, the dynamics refers to calculating transitions between states in a mode that defines the state updating policy. The full dynamics computation is available in **BooN** in both the

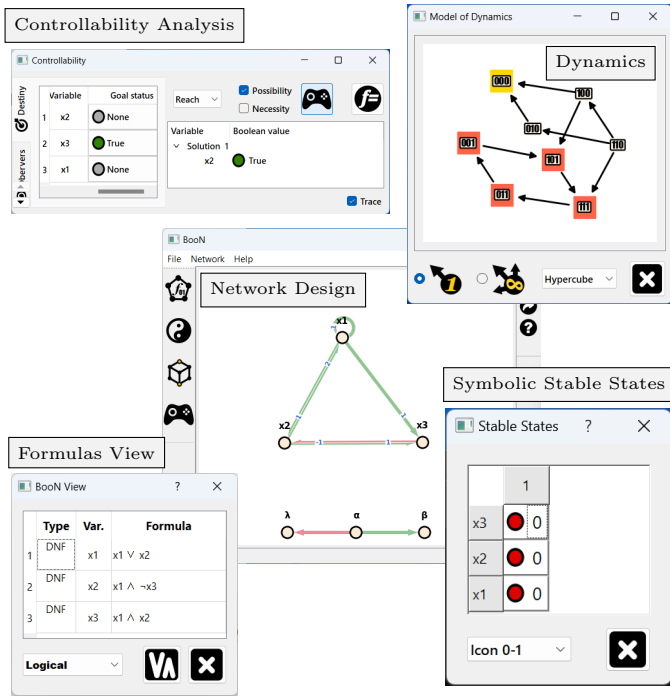


Fig. 4: Overview of the **BooN** functionalities/windows using GUI.

synchronous and asynchronous modes, and more generally for any possible mode.

The symbolic computation of stable states efficiently lists the interpretations of the following equivalence stating the fixed point of the dynamics: $f(x) \iff x$. The resolution is achieved by a SAT solver, specifically the Z3-Solver.

Moreover, the controllability analysis entails the computation of prime implicants, as detailed in the reference (Biane and Delaplace, 2019). These prime implicants are used to extract the active control parameters which define the control to be applied.

Given that a control is active if it is equal to 0 ($u_i^c = 0$), we extract the negative control parameters from the prime implicants, which are true only when they are set to 0, for the purpose of defining the core, which is the minimal set of active control parameters under the inclusion. The other variables, positive controls and network variables, are discarded since they do not share information related to the control activity. The prime implicant algorithm that has been implemented is based on the translation of the problem into an integer linear programming problem (Pizzuti, 1996) and the integer linear programming solver used is PULP.

4. Method performance

The primary methods of **BooN** are benchmarked using randomly generated Boolean networks. We classified these networks into two categories based on the topology of their interaction graphs: Erdős-Rényi and Small-World. The formulas were constructed from interaction graphs defining variable dependencies. For each variable, a DNF formula is randomly generated from the input neighbors of the variable within the interaction graph. The number of clauses is randomly defined within predefined bounds, and the negations are determined from a parameter controlling the likelihood of positive terms. Formulas derived from the Erdős-Rényi

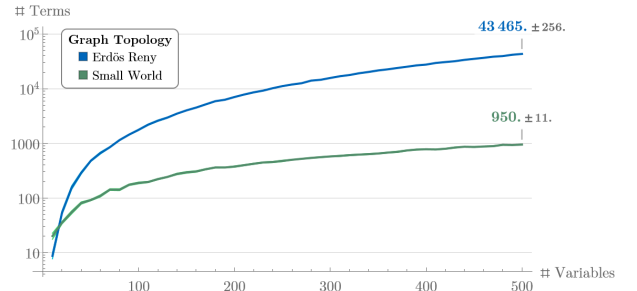


Fig. 5: Number of terms per network size and topology.

interaction graph proved more complex, containing a greater number of terms than those from the Small-World interaction graph (Figure 5). Consequently, the benchmark analysis is divided into two formula categories: complex formulas (Erdős-Rényi) and simpler formulas (Small-World). The benchmark was computed on a INTEL Icore 7 (1.90 Ghz) with 16Go of RAM.

The benchmark assesses the efficiency of the methods on networks of increasing size in terms of the number of variables. For each size, 10 networks are randomly generated. The curves show the average for each size with its uncertainty. The benchmark Jupyter program is included in the GitHub release. We respectively assess the stable state computation, the brute force equilibria computation and the controllability analysis.

4.1. Stable states

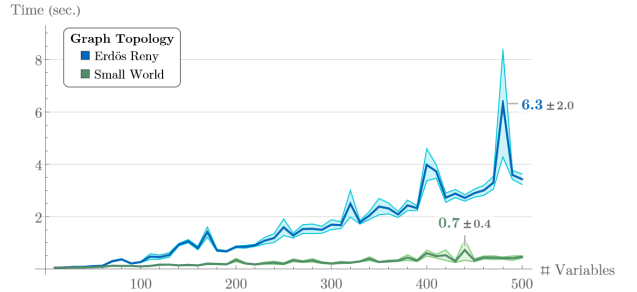
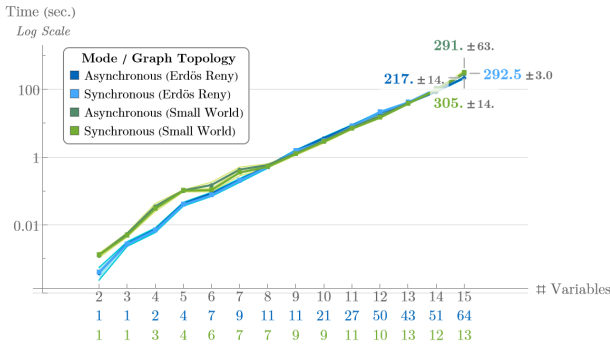


Fig. 6: Time for stable states computation.

The stable state method based on symbolic resolution using the Z3-solver is a very fast and scalable approach, with processing times of less than 6.3 sec for large networks comprising thousands of terms (Figure 5). The complexity of the formulas has a significant impact of the time computation.

4.2. Equilibria

The brute force algorithm characterizing the attractors is performed on the whole state space ($2^{|X|}$ states). As might be expected, the time required is doubled with each additional variable, and thus follows an exponential growth curve. Nevertheless, the algorithm has been optimized to remain efficient when considering a limited number of variables, which in this case is 15. Our analysis revealed no notable discrepancies between the Small-World and Erdős-Rényi formulas. This is because the primary constraint stems



Below the number of variables is the number of terms for the Erdos-Reny and Small-World networks.

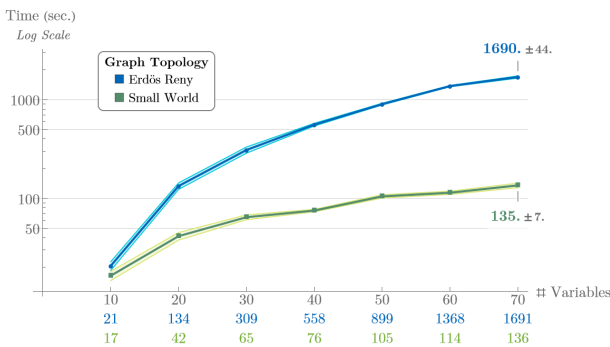
Fig. 7: Time for equilibria computation.

from the exponential number of states, rather than the complexity of the formulas themselves.

4.3. Controllability

The final benchmark pertains to the evaluation of controllability. The queries ascertain some requisite conditions corresponding to a Boolean profile for the variables x_0 and x_1 . The selected profile is never satisfied in the current stable states. The goal is to possibly reach this profile at stable state (in 1 stable state at least).

This benchmark demonstrates that the controllability method is efficient for the typical Boolean networks designed to model genetic networks, the majority of which have around 30 nodes and rarely exceed 50. However, the performance is contingent on the complexity of the formulas, thereby explaining the observed time difference between Erdős-Renyi and small-world networks. Indeed, the number of terms has a direct influence on the resolution capability of the PULP ILP-solver, since the system to be solved will be more complex.



Below the number of variables is the number of terms for the Erdős-Reny and Small-World networks.

Fig. 8: Time evaluation for controllability (possibility) queries.

5. Conclusion

BooN provides a python package for Boolean network analysis and a graphical interface which provides a convenient way to visualize and interact with the network. It also offers unique capabilities for controllability analysis, which is not available in other software.

A number of software applications are devoted to controllability analysis, as CABEAN (Su and Pang, 2021) or Bonesis (Chevalier et al., 2024). However, they are focused on this specific analysis and do not encompass the other aspects related to Boolean dynamics analysis.

In conclusion, **BooN** is a software solution tailored for Boolean network dynamics analysis, featuring a unique ability to conduct controllability analysis. Furthermore, its graphical interface allows for the straightforward manipulation of Boolean networks.

References

- Beneš, N., Brim, L., Huvar, O., Pastva, S., Šafránek, D., and Šmíjáčková, E. (2022). AEON.py: Python library for attractor analysis in asynchronous Boolean networks. *Bioinformatics*, 38(21):4978–4980.
- Biane, C. and Delaplace, F. (2019). Causal Reasoning on Boolean Control Networks Based on Abduction: Theory and Application to Cancer Drug Discovery. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 16(5):1574–1585.
- Chevalier, S., Boyenval, D., Magaña-López, G., Roncalli, T., Vaginay, A., and Paulevé, L. (2024). Bonesis: a python-based declarative environment for the verification, reprogramming, and synthesis of most permissive boolean networks. In Gori, R., Milazzo, P., and Tribastone, M., editors, *Computational Methods in Systems Biology*, pages 71–79, Cham. Springer Nature Switzerland.
- Glass, L. and Kauffman, S. A. (1973). The logical analysis of continuous, non-linear biochemical control networks. *Journal of Theoretical Biology*, 39(1):103–129.
- Kauffman, S. (1969). Homeostasis and differentiation in random genetic control networks. *Nature*, 224:177–178.
- Klärner, H., Streck, A., and Siebert, H. (2017). PyBoolNet: A python package for the generation, analysis and visualization of boolean networks. *Bioinformatics*, 33(5):770–772.
- Müssel, C., Hopfensitz, M., and Kestler, H. A. (2010). BoolNet—an R package for generation, reconstruction and analysis of Boolean networks. *Bioinformatics*, 26(10):1378–1380.
- Naldi, A., Berenguier, D., Fauré, A., Lopez, F., Thieffry, D., and Chaouiya, C. (2009). Logical modelling of regulatory networks with GINsim 2.3. *Bio Systems*, 97(2):134–139.
- Naldi, A., Hernandez, C., Levy, N., Stoll, G., Monteiro, P. T., Chaouiya, C., Helikar, T., Zinovyev, A., Calzone, L., Cohen-Boulakia, S., Thieffry, D., and Paulevé, L. (2018). The CoLoMoTo Interactive Notebook: Accessible and Reproducible Computational Analyses for Qualitative Biological Networks. *Frontiers in Physiology*, 9.
- Oles, V. and Kukushkin, A. (2020). BoolSi: A Tool for Distributed Simulations and Analysis of Boolean Networks. *Journal of Open Research Software*, 8(1).
- Pardo, J., Ivanov, S., and Delaplace, F. (2021). Sequential reprogramming of biological network fate. *Theoretical Computer Science*, 872:97–116.
- Pizzuti, C. (1996). Computing prime implicants by integer programming. In *Proceedings of the 8th International Conference*

-
- on Tools with Artificial Intelligence*, ICTAI '96, page 332, USA.
IEEE Computer Society.
- Su, C. and Pang, J. (2021). CABEAN: A software for the control of asynchronous Boolean networks. *Bioinformatics*, 37(6):879–881.
- Thomas, R. (1973). Boolean formalization of genetic control circuits. *Journal of Theoretical Biology*, 42(3):563–585.